

Software Engineering

CE2006/CZ2006

Requirements Elicitation 2 of 3

Discussion Topics

- Use Case modeling
- Use Case diagram
- How to develop Use Cases from functional requirements
- Reading
 - Bruegge Chap 4.4
 - Fowler Chap 9

UML: Unified Modeling Language



- Created by Booch, Jacobson & Rumbaugh in 1996
- Version 1.1 was adopted by the Object Management Group (OMG) in 1997. The latest version is 2.5 (August 2015).
- What is it?
It is a **graphical notation with textual annotation** for specifying, documenting and communicating various aspects of the **structure, functionality** and **dynamic behaviour** of complex software systems.

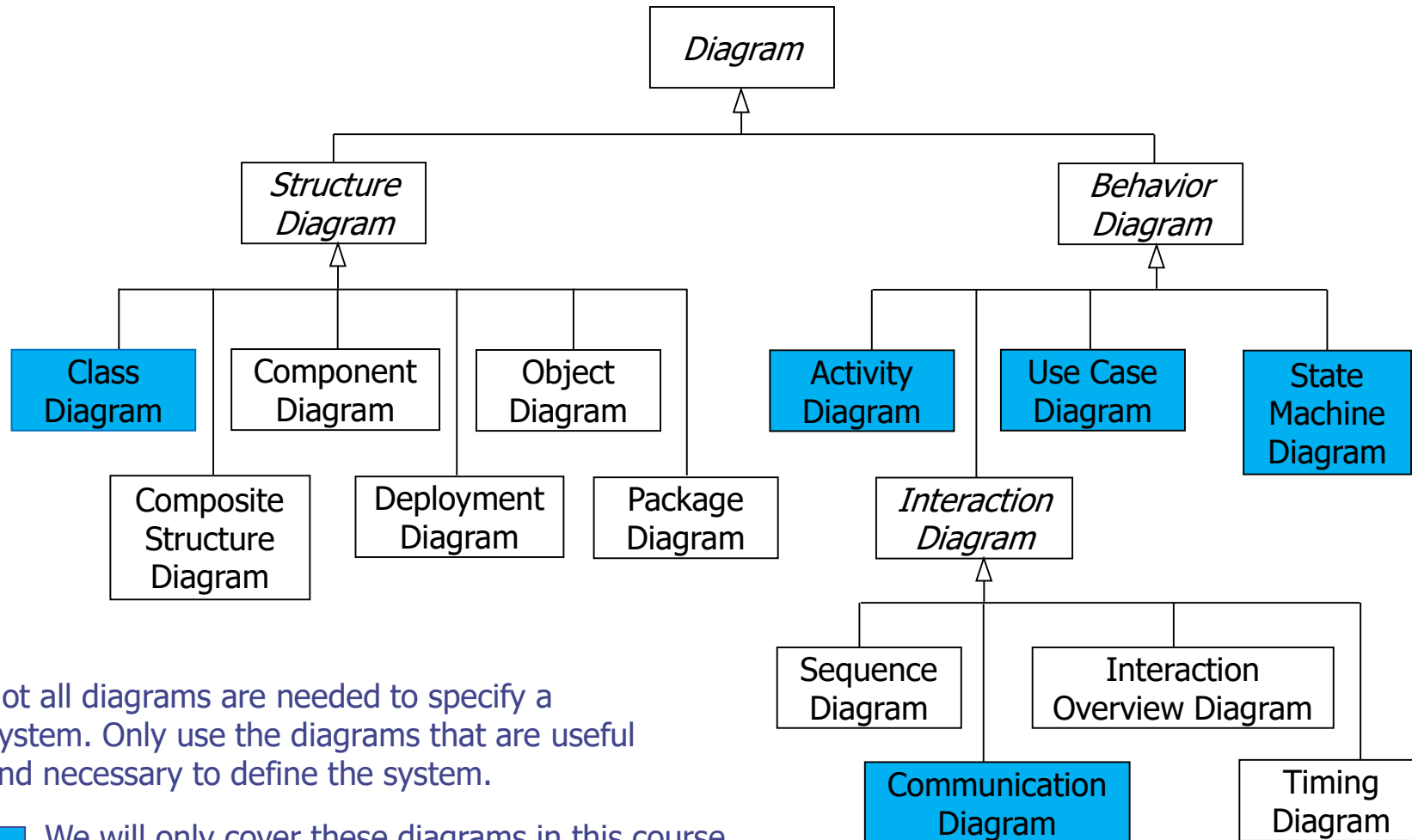
It is **NOT a programming language** (although its supporters are working towards that goal).

It is **NOT the answer to all your modelling and design problems.**

UML is an Industry Standard tool and therefore Provides Standardization!

In the laboratory you can use Visual Paradigm to draw UML diagrams for your project work.

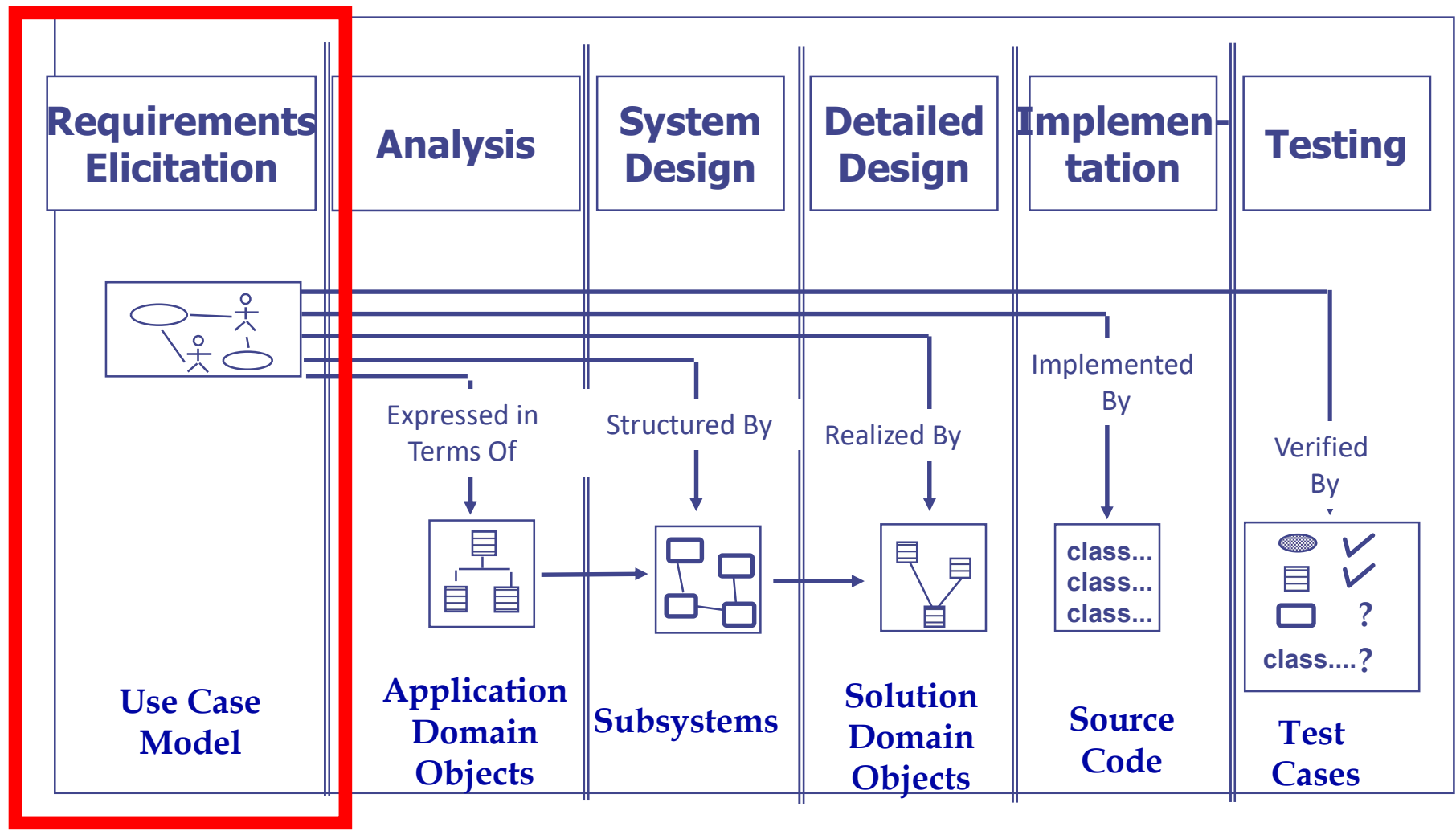
UML Diagram Types



Not all diagrams are needed to specify a System. Only use the diagrams that are useful and necessary to define the system.

 We will only cover these diagrams in this course (Others will be covered in CZ3003)

Software Development Lifecycle Activities



Use Cases

- A use case is a software and system engineering term that describes how a user uses a system to accomplish a particular goal. – Google
- A use case is a list of actions or event steps typically defining the interactions between a role (known as an actor) and a system to achieve a goal. – Wikipedia

Functional Requirements versus Use Cases

- Both are about system functionalities
 - Functional requirements: **what** (user, external system, functionality, and information)
 - Use cases: **how** (actors **interact** with system functionality)

Functional requirements are the starting points for use case modeling.

Use Case Model

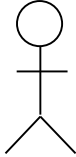
Use Case Diagram (static)

+

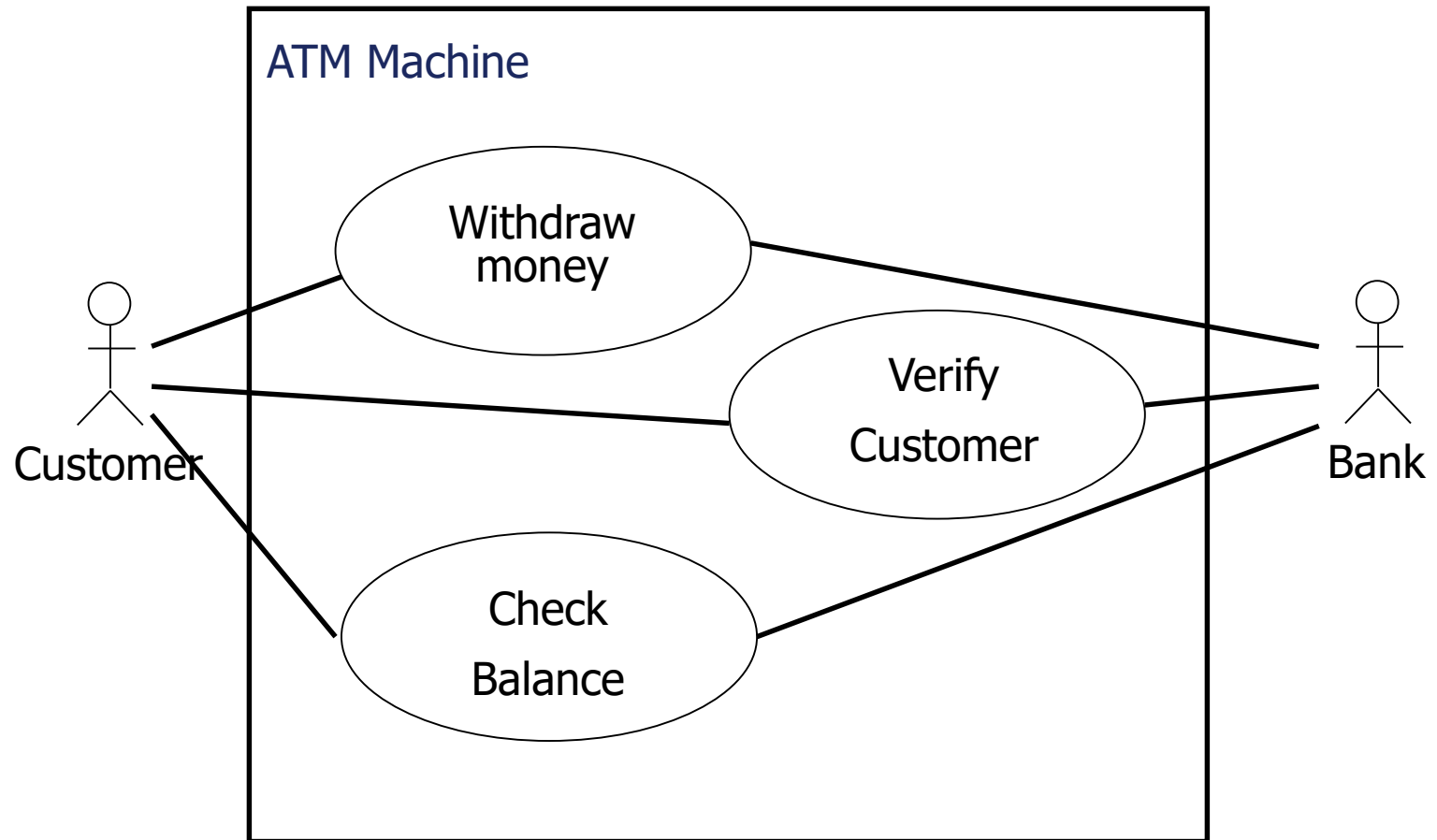
Use Case Description (dynamic)

- Provides coherent visual and textual description of system functionalities
- In use-case-driven iterative development, prioritise use cases for implementation

Use Case Diagram Elements

Name	Notation	Description
Actor		An external entity that interacts with system
Use Case		Unit of functionality performed by system, which yields result / value for Actor
Association		Connects Actor to Use Cases(s) in which they participate

Example of Use Case Diagram

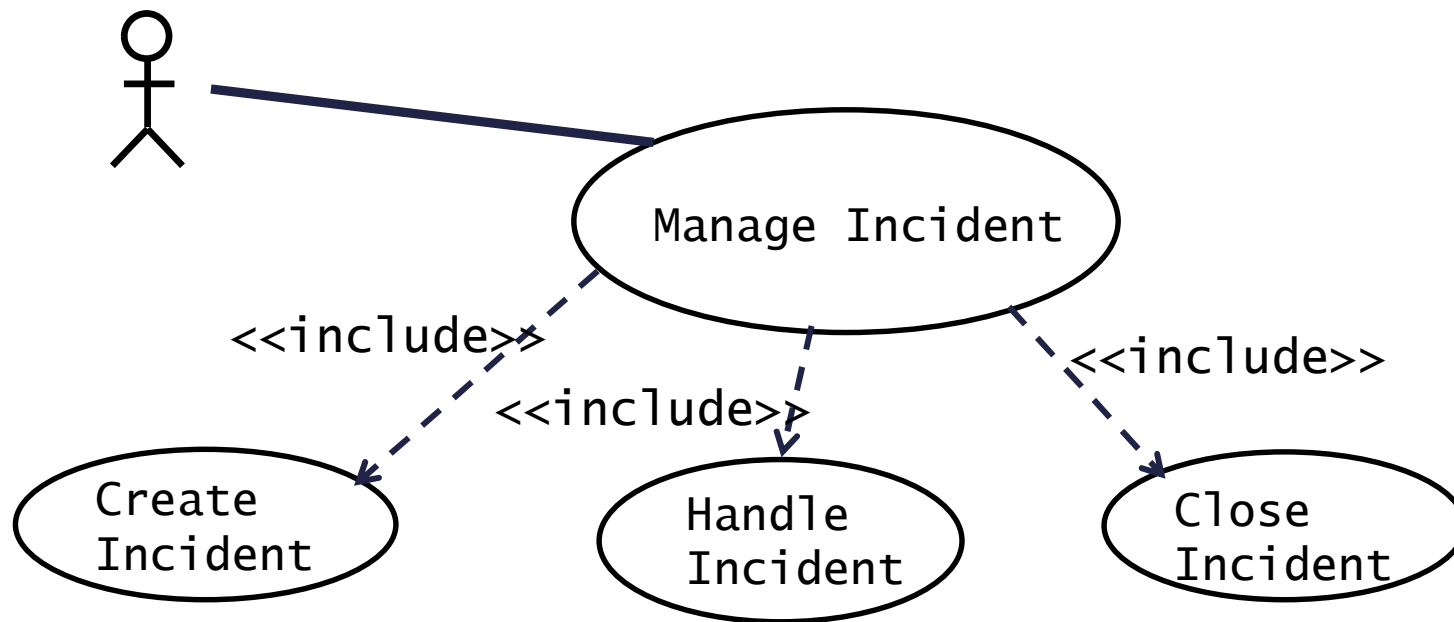


Use Case Associations

- Dependencies between use cases are represented by **use case associations**
- Associations are used to reduce complexity
- Two types of use case associations
 - **Includes**
 - **Extends**

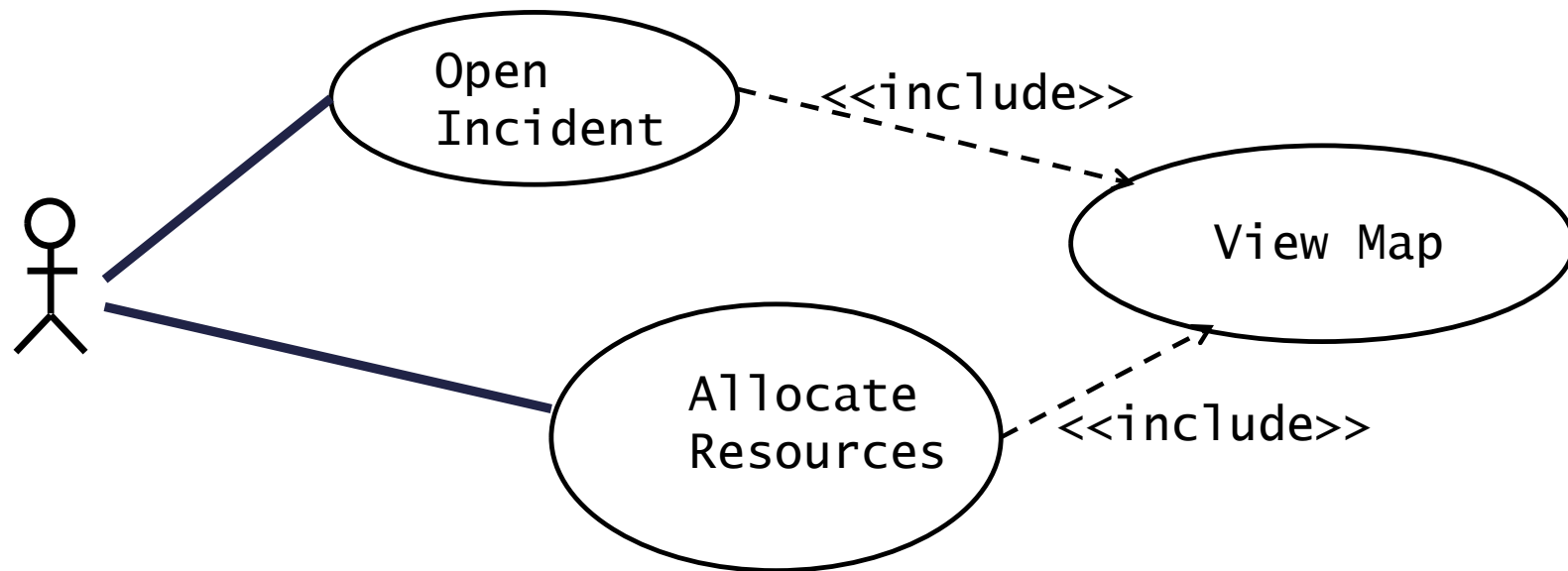
<<include>>:Functional Decomposition

- **Problem:** A function in the original problem statement is too complex
- **Solution:** Describe the function as the aggregation of a set of simpler functions. The associated use case is decomposed into shorter use cases



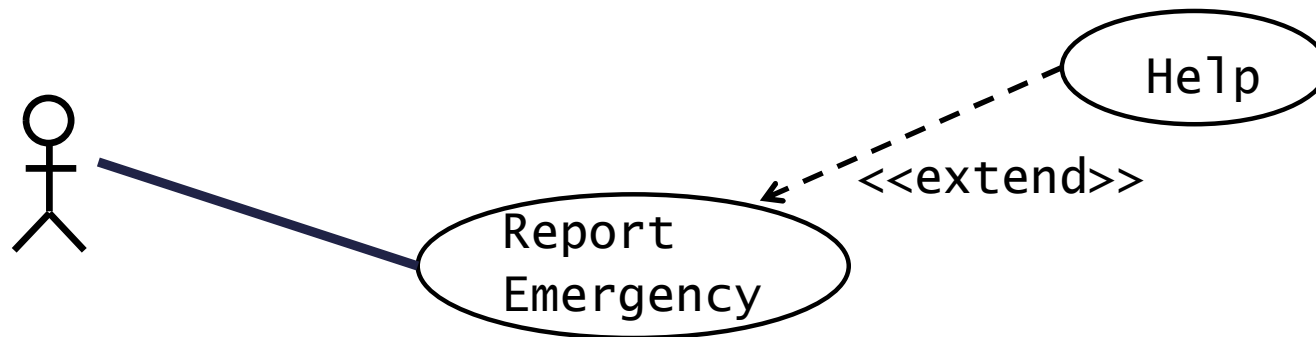
<<include>>: Reuse Existing Functionality

- Problem: There are overlaps among use cases.
- Solution: The ***include association*** from use case A to use case B indicates that an instance of use case A performs all the behaviour described in use case B ("A delegates to B")
- Example: Use case "View Map" describes behaviour that can be used by use cases "Open Incident" and "Allocate Resources"



<<extend>>: Adding Optional Functionality

- Problem: The functionality in the original problem statement needs to be extended.
- Solution: An *extend association* from use case A to use case B
- Example: "ReportEmergency" is complete by itself, but **can be extended** by use case "Help" for a scenario in which the user requires help



Include, extend associations

- Include = **reuse** of functionality
 - denotes a dependency on another use case. Included Use Cases are always used in a parent Use Case.
- Extends = **adding** (optional) functionality. Extends are only used in the parent Use Case in exceptional or unusual circumstances.

▪

Example Functional Requirements of a Library Management System

1. The library member must be able to search the system for library materials
2. The library member must be able to loan library materials
3. The library management system must verify the library membership with the University Account System before the library member can loan library materials
4. The library member must return library materials via library dropoff kiosk
5. The library management system must send overdue notice using the CITS Email Server to the library member
6. When there are network issues to connect to CITS Email Server, the library management system must log the event

Identify Potential Actors

1. The **library member** must be able to search the system for library materials
2. The library member must be able to loan library materials
3. The library management system must verify the library membership with the **University Account System** before the library member can loan library materials
4. The library member must return library materials via **library dropoff kiosk**
5. The library management system must send overdue notice using the **CITS Email Sever** to the library member
6. When there are network issues to connect to CITS Email Server, the library management system must log the event

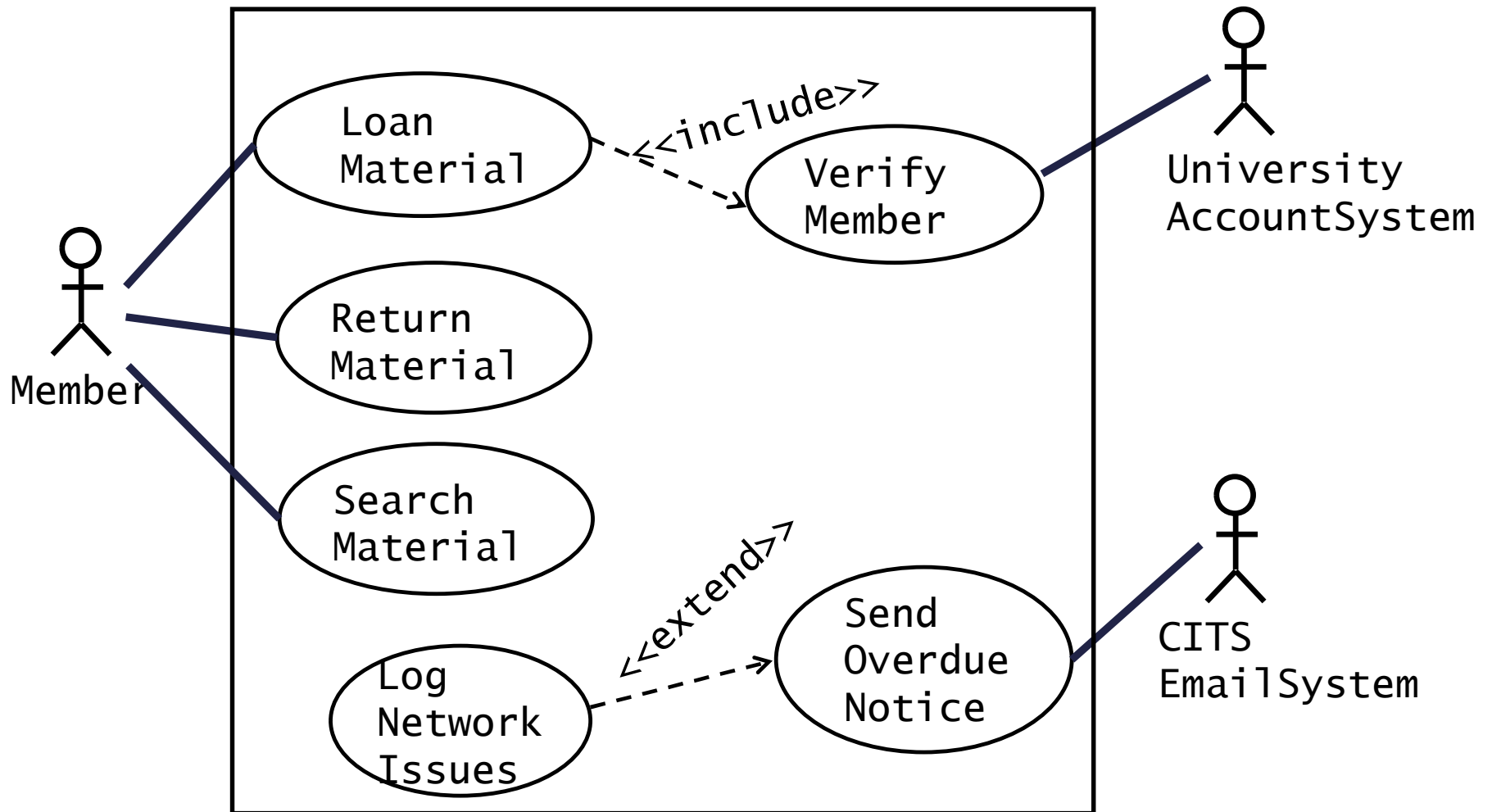
Identify Potential Use Cases

1. The **library member** must be able to **search the system** for library materials
2. The library member must be able to **loan library materials**
3. The library management system must **verify the library membership** with the **University Account System** before the library member can loan library materials
4. The library member must **return library materials** via **library dropoff kiosk**
5. The library management system must **send overdue notice** using the **CITS Email Sever** to the library member
6. When there are network issues to connect to CITS Email Server, the library management system must **log the event**

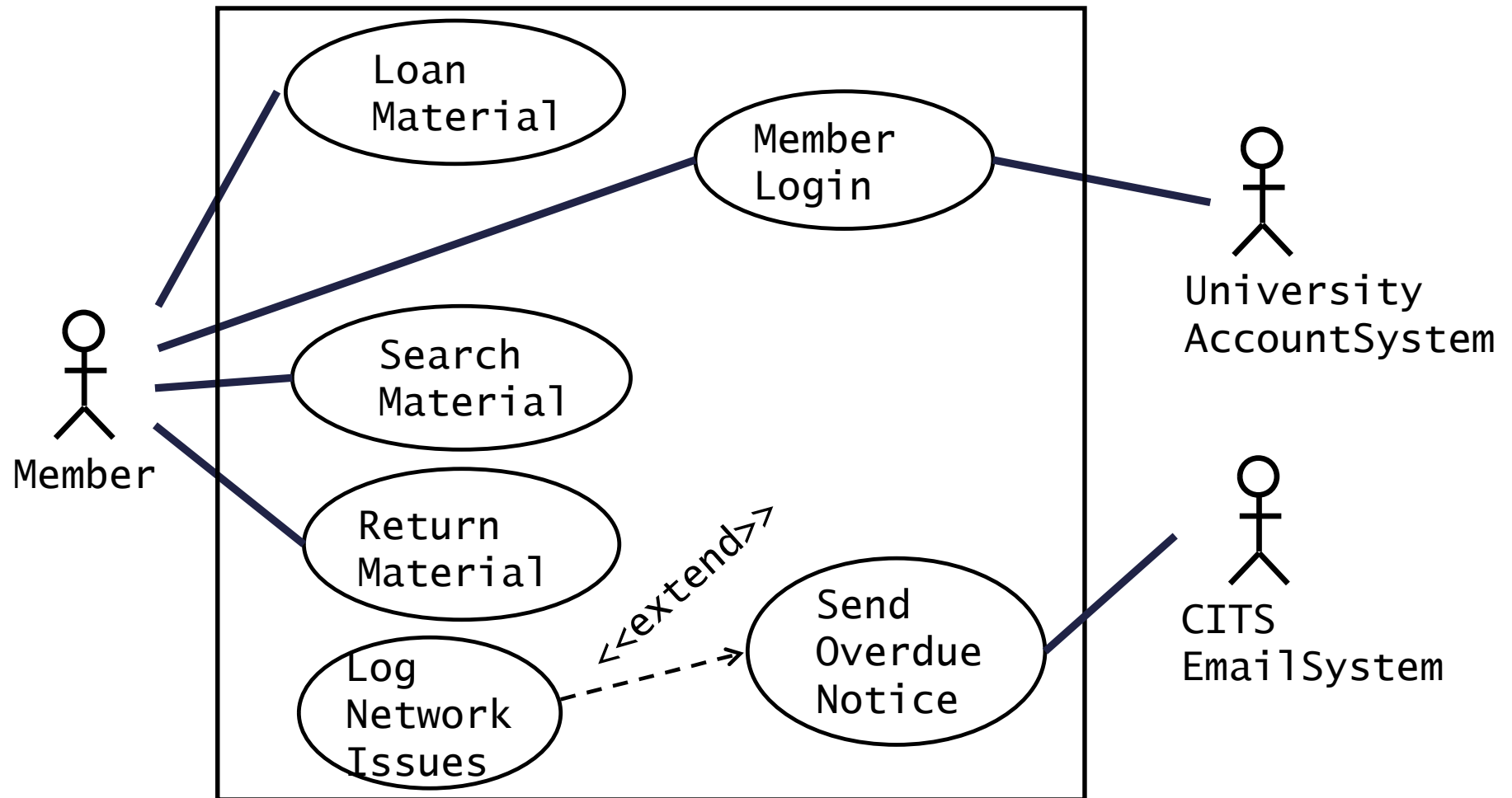
Identify Potential Associations Between Use Cases

1. The **library member** must be able to search the system for library materials
2. The library member must be able to loan library materials
3. The library management system must verify the library membership with the **University Account System** **before** the library member can loan library materials
4. The library member must return library materials via **library dropoff kiosk**
5. The library management system must send overdue notice using the **CITS Email Sever** to the library member
6. **When** there are **network issues** to connect to CITS Email Server, the library management system must log the event

Use Case Diagram 1



Use Case Diagram 2



Use Case Diagram 3

